MLPilot

AI-Powered ML Experimentation Agent

An intelligent system that takes a tabular dataset, investigates its quality, plans machine-learning experiments, executes approved tools, compares model performance, and explains the results using evidence.

Machine Learning	Agentic AI	LangChain	LangGraph	Production Deployment

Contents

#	Section
1	Complete workflow and design principle
2	Module-wise project plan (updated with LangChain)
3	Updated architecture and LangChain usage
4	Why we need Modules 3 and 4 (LangChain and LangGraph)
5	Version-wise roadmap
6	14-day schedule
7	How we will work together
8	What we do next

1 Complete workflow

User uploads a CSV
Dataset and target column

▼

Module 1 — Data Validation
Check the input is usable

▼

Module 2 — Dataset Intelligence & ML Pipeline
Profile quality, understand the target, preprocess, train, evaluate

▼

Module 3 — AI Experiment Planner
LLM chooses from permitted experiments

▼

Module 4 — Agent Orchestration
LangGraph coordinates tools, routing and recovery

▼

Module 5 — Experiment Comparison
Compare actual metrics and rank runs

▼

Module 6 — AI Results Explainer
Explain findings based on measured results

▼

Module 7 — Dashboard and Experiment History
Visualize, review and export results

Core design principle: the ML engine calculates the results; the AI agent coordinates the workflow and interprets the results. The LLM must not invent model scores or run arbitrary generated code.

2 Module-wise project plan

Final module plan, with LangChain and LangGraph built in. Each module has a clear deliverable so something usable exists at every stage.

Module 0 — Project Setup

Foundation

Purpose: Prepare the development environment and project structure.

- Create the GitHub repository and Python virtual environment.
- Organize folders and dependencies.
- Configure environment variables and Git.
- Understand how modules will communicate.

Deliverable: A clean, runnable repository.

Module 1 — Dataset Validation and Profiling

ML foundation

Purpose: Understand incoming data before training a model.

- Load CSV files using Pandas.
- Inspect rows, columns and data types.
- Detect missing values, duplicate rows and unusual values.
- Validate the target column and input size.
- Return profiling results in a structured format.

Deliverable: A reusable Python data-profiling module.

Concepts: Pandas, functions, dictionaries, exceptions, data validation.

Module 2 — ML Pipeline Engine

Core ML

Purpose: Convert raw data into trained, evaluated models.

- Identify classification versus regression tasks.
- Separate features from the target.
- Split data into training and testing sets.
- Handle missing values, categorical features and scaling where needed.
- Train a small set of appropriate baseline models.
- Calculate task-specific evaluation metrics.

Deliverable: A reusable engine that trains and evaluates models without an LLM.

Concepts: Scikit-learn, pipelines, preprocessing, model evaluation, leakage prevention.

Module 3 — AI Experiment Planner (LangChain)

GenAI foundation

Purpose: Let an LLM propose sensible experiments for a dataset.

- Connect a LangChain chat model and build prompt templates.
- Send a concise dataset profile to the LLM.
- Ask it to select a task type and candidate models.
- Request structured output (Pydantic schemas) rather than free-form instructions.
- Validate the proposed plan against an allowlist; reject unsupported recommendations.

Deliverable: An AI-generated experiment plan that your application can validate.

Concepts: Prompt engineering, structured output, tool calling, hallucination control.

Module 4 — Agent Orchestration (LangChain tools + LangGraph)

Core agent engineering

Purpose: Coordinate the ML workflow through LangGraph.

- Define a typed workflow state.
- Create planner, execution and reporting nodes.
- Register deterministic Python functions as LangChain tools.
- Add conditional routing and failure handling.
- Pass actual results between nodes.

Deliverable: A functioning agent that plans experiments and invokes ML tools through a controlled workflow.

Concepts: LangGraph state, nodes, edges, tool calling, workflow control.

Module 5 — Experiment Tracking and Comparison

Reproducibility

Purpose: Make experiment results reproducible and comparable.

- Store model name, configuration, seed and metrics.
- Compare experiments using an appropriate primary metric.
- Rank completed runs and handle failed runs.
- Persist experiment metadata in SQLite.

Deliverable: A reliable experiment history with reproducible comparisons.

Module 6 — Results Explanation

Evidence-based reporting

Purpose: Explain what the results mean in plain language, grounded in actual metrics.

- Summarize measured metrics.
- Explain why the selected model ranks highest on the chosen metric.
- Report limitations and data-quality concerns.
- Optionally add permutation feature importance.

Deliverable: An evidence-based experiment report, not an unsupported AI-generated conclusion.

Module 7 — API, Dashboard and Deployment

Delivery

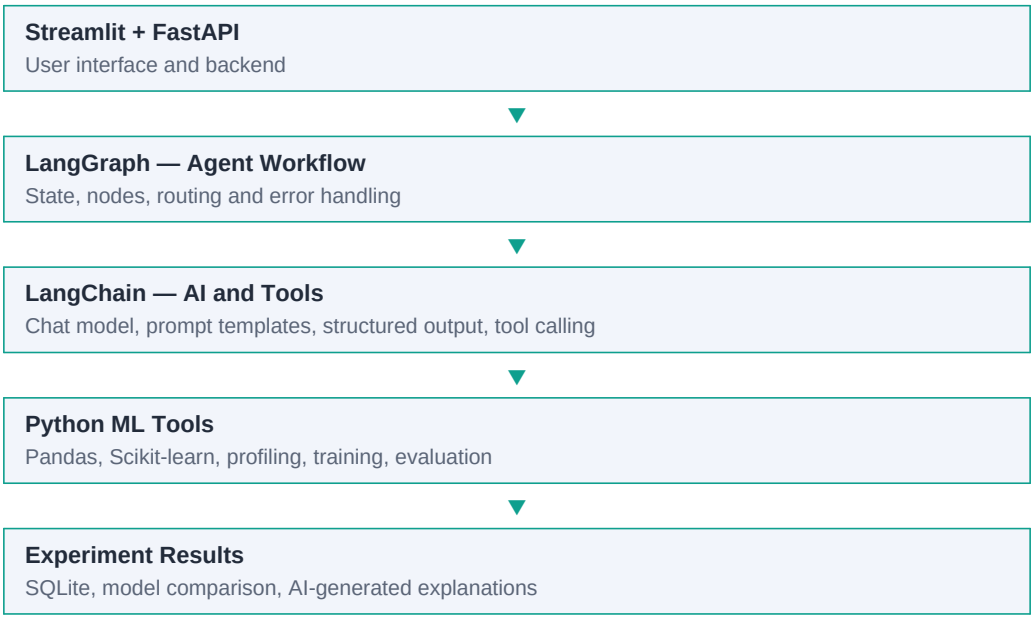
Purpose: Turn your Python modules into a usable application.

- Build API endpoints with FastAPI.
- Create an upload and experiment dashboard using Streamlit.
- Show profiling, experiment metrics and result explanations.
- Test the integrated application.
- Containerize with Docker and deploy a public demo.

Deliverable: A live application that another person can use.

3 Updated architecture and LangChain usage

LangChain is now a core part of MLPilot, used for LLM integration, prompts, structured outputs and tool calling. LangGraph handles workflow orchestration. They serve different purposes, so using both is appropriate. No separate agent framework is added.



Where you will use LangChain

LangChain concept	Implementation in MLPilot
Chat models	Connect to a supported LLM provider
Prompt templates	Generate experiment-planning prompts
Structured output	Produce validated experiment plans
Tools	Expose Python profiling, training and comparison functions
Tool calling	Let the LLM select from permitted actions
Output validation	Validate plans using Pydantic schemas

Scope note: the two-week schedule stays the same. Prioritize the core features and keep advanced additions optional.

4 Why we need Modules 3 and 4

Question: If Module 2 already trains ML models and gives us the results, why do we need Module 3 (LangChain) and Module 4 (LangGraph)? **Answer:** Module 2 builds the ML engine; Modules 3 and 4 make it intelligent and capable of coordinating experiments.

4.1 Two versions of MLPilot

Version A — ML pipeline without AI	Version B — MLPilot (ML + LangChain + LangGraph)
1. User uploads dataset 2. Python trains 3 predefined models 3. Display metrics and best model **Limitation:** always follows the same predefined process, regardless of what the dataset reveals.	1. User uploads dataset 2. **LangChain:** LLM interprets the profile and proposes experiments 3. **LangGraph:** coordinates planning, validation, execution and recovery 4. **Module 2:** executes experiments and calculates real metrics 5. Results, comparison and evidence-based explanation

4.2 A real example: customer churn

A user uploads 10,000 customer records and wants to predict whether a customer will leave. The target column is **Churn**, and only 5% of customers have churned.

What Module 2 does: splits the data, preprocesses features, trains Logistic Regression, Decision Tree and Random Forest, then calculates precision, recall, F1-score and other metrics. Illustrative results:

Model	Accuracy	Recall for churn
Logistic Regression	95%	20%
Decision Tree	92%	45%
Random Forest	95%	35%

Illustrative only. On accuracy alone, Logistic Regression and Random Forest look excellent, yet they miss many customers who actually churn.

What Module 3 adds (LangChain): the LLM interprets the profile and might recommend: *the target is imbalanced; compare models using recall and F1-score in addition to accuracy; consider class-weighted training and validate with an appropriate split.* It returns a structured plan that the application validates before execution. Module 3 does not replace Module 2: it determines what experiments are worth attempting, and the ML engine performs them.

What Module 4 adds (LangGraph): manages the execution flow:

- Receive the dataset profile.
- Ask the planner for an experiment plan.
- Validate the plan.
- Call the ML training tool.
- Evaluate the results.
- If an experiment fails, handle the failure or try an allowed alternative.
- Produce the final report.

LangGraph is valuable when there are multiple steps, conditional routing, state and failure handling. For a simple linear workflow it may be unnecessary; it is included here to demonstrate meaningful agent orchestration.

4.3 Do we actually need both?

Component	Responsibility	Works without AI?
Module 2: ML engine	Train models and calculate metrics	Yes
Module 3: LangChain	Interpret context and propose experiments	Yes, but AI planning is lost
Module 4: LangGraph	Coordinate workflow, routing and recovery	Yes, via ordinary Python orchestration

Layer	Question it answers
ML engine	What happened when we trained the model?
LangChain	What should we try, given what we know?
LangGraph	What step should run next, and how should the workflow respond?

> **Important limitation:** an LLM's recommendations are not automatically correct. We validate them, and actual ML metrics decide whether a new experiment improves performance.

4.4 Recommendation for the two-week project

- Build Module 2 first, but keep it modular. Then add LangChain and LangGraph around it.
- Do not over-engineer the ML pipeline: start with a few baseline models.
- Let the agent choose among a limited set of valid experiments.
- The result is a genuine ML application with a meaningful agentic AI layer, not a training dashboard with a chatbot attached.

5 Version-wise roadmap

Each version is a usable milestone: something works at the end of each stage rather than integrating everything on the last day. Versions are development milestones, not separate products. Keep scope small enough that v1.0 is achievable in two weeks.

Version	Days	Focus	Milestone
v0.1	1–2	**Dataset Inspector** Load, validate, inspect data types, detect missing values and duplicates.	Data profiling works independently.
v0.2	3–4	**ML Engine** Train baseline models and calculate appropriate metrics.	ML works without any AI dependency.
v0.3	5–6	**AI Planning** Generate and validate experiment plans using an LLM.	The AI produces valid, executable plans.
v0.4	7–8	**Agent Workflow** Connect planning, tool execution, evaluation and reporting using LangGraph.	An end-to-end agent run succeeds.
v0.5	9–10	**Interactive Application** Experiment history, model comparison charts and results dashboard.	Users can run and inspect experiments.
v1.0	11–14	**Tested and Deployed MLPilot** Integration tests, error-handling hardening, Docker deployment, documentation.	Portfolio-ready app with a public demo.

6 14-day schedule

Day	Main task	Expected result
1	Project structure and requirements	Architecture and setup ready
2	Dataset profiling	First tested module
3	Preprocessing	Clean ML input pipeline
4	Training and metrics	Working baseline models
5	LLM planner	Structured experiment plan
6	Tool definitions and validation	Safe, callable ML tools
7	LangGraph state and nodes	Initial agent workflow
8	Routing and error handling	End-to-end workflow
9	Experiment history	Stored metrics
10	Streamlit dashboard	Interactive application
11	Unit tests	Core functions tested
12	Integration and edge cases	Reliable workflow
13	Docker and deployment	Hosted application
14	README, demo and resume	Portfolio-ready deliverable

7 How we will work together

Since you want to write the code yourself, every module follows the same process:

Step	What happens
1. Understand	I explain what the module does and why the project needs it.
2. Design	We define inputs, outputs, functions and dependencies before coding.
3. Break into tasks	You receive small implementation challenges, not a complete code dump.
4. Implement	You write the code yourself and share it when ready.
5. Review	I help debug, explain errors and ask questions to check understanding.
6. Test and integrate	We verify the module and connect it to previous modules before moving on.

Working rule: I explain the concept, define the module contract, and give you a small task. You write the code; we review and test it together.

8 What we do next

Finalize requirements and design for **Module 0: Project Setup** and **Module 1: Dataset Validation and Profiling** before any coding.

- Write the project requirements document.
- Define the exact input/output contract for each module.
- Begin implementation one small task at a time.

Scope decision: the first version is a **single-agent workflow with deterministic ML tools**, not a multi-agent system. This keeps the project manageable while still demonstrating meaningful agent engineering.